DESIGN PATTERNS · BEHAVIORAL | GoF · 15 / 23 | 4-dive study card

Interpreter

Given a language, define a class representation for its grammar plus an interpreter that evaluates sentences in it.

Airplane porthole — DIVE 01 · ORIENTATION

▼ 10 km

Each grammar rule becomes a class — sentences turn into a syntax tree of these objects that you evaluate by walking it.

— THE CONTOURS

- **Grammar as classes** — one Expression class per rule, terminal or composite
- **Sentence = a tree** — the AST is a Composite of expression objects
- **interpret(context)** — each node evaluates itself, recursing into children

— USE IT WHEN — GoF applicability

- **A simple, stable grammar** — recurs often and is worth representing as a class tree
- **Sentences map cleanly** — to an abstract syntax tree (and efficiency isn't critical)

Behavioral family: Interpreter walks a grammar tree · Visitor adds operations over it · Iterator traverses it · the AST itself is a Composite. Rarely hand-built today.

DIVE DEEPER

Train window — DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

- **AbstractExpression** — declares interpret(context) for every node
- **TerminalExpression** — a leaf — literals, variables, the atoms of the grammar
- **NonterminalExpression** — a composite rule (And, Or, Plus) holding sub-expressions

— IN THE WILD

- **Regular expressions** — a pattern compiled to a matcher tree — the classic interpreter
- **SQL / query parsers** — a WHERE clause becomes an evaluable expression tree
- **Rules / spec engines** — business rules as composable boolean expressions
- **Template & DSL engines** — Liquid, SpEL evaluate expression nodes

`parse(text) → an Expression tree → tree.interpret(context) evaluates it bottom-up.`

> **GoF intent:** "Define a representation for a grammar plus an interpreter that uses it to interpret sentences."

DIVE DEEPER

Park walk — DIVE 03 · CONNECTIONS

▼ 100 m

Sentence (raw text) → **Parser** (builds the AST) → **Expression tree** (Composite of rules) → ✓ **interpret(ctx)** (evaluated)

— WHY IT FOLLOWS

- → **One class per rule ⇒** the grammar's structure maps directly to a class hierarchy
- → **AST is a Composite ⇒** terminals are leaves, rules are nodes with children
- → **Recursive interpret() ⇒** evaluating the root walks the whole tree

— STRUCTURE · UML

Expression

- `- children: Expression[]`
- `+ interpret(ctx): Result`

Terminals are leaves; Nonterminals hold child expressions and recurse. interpret() walks the AST to a result.

— WHAT IT BUYS YOU

- ✓ **Easy to change grammar —** rules are classes — extend by adding a class
- ✓ **Easy to implement —** each rule's interpret() is small and local
- ✓ **Grammar is explicit —** the class tree documents the language
- ✓ **Doesn't scale —** complex grammars explode into too many classes
- ✓ **Add ops via Visitor —** new operations without touching the node classes

The AST is a Composite; Visitor adds operations over the tree without changing nodes. Flyweight can share terminal symbols, Iterator traverses the tree, and a Builder / parser constructs it.

DIVE DEEPER

Microscope — DIVE 04 · DETAILS

▼ 1 cm

> **Definition:** "Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language."
> — GoF, "Design Patterns", 1994

— THE CODE

```
interface Expr { interpret(ctx): boolean }
class Var implements Expr {
constructor(private n) {}
interpret(c) { return c[this.n]; }
}
class And implements Expr {
constructor(private l, private r) {}
interpret(c) {
return this.l.interpret(c) && this.r.interpret(c);
}
}
```

— VARIANTS

- **Terminal expression** — leaf nodes — literals, variables, constants
- **Nonterminal expression** — composite rules — And, Or, Plus hold sub-exprs
- **Context object** — carries variable bindings & input to interpret()
- **Parser + Interpreter** — build the AST, then evaluate it (two phases)
- **Internal DSL** — fluent builders form the tree in host code
- **Specification** — composable boolean business rules — interpreter kin

— COMPARE THE ALTERNATIVES

Property	Interpreter	Visitor	Composite	Parser-gen
Models a grammar	✓	✗	~	✓
Evaluates sentences	✓	~	✗	~
Adds ops without edits	✗	✓	✗	~
Builds the tree	✗	✗	✗	✓
Scales to big grammars	✗	✓	✓	✓

Interpreter evaluates a small grammar · Visitor adds operations · Composite is the tree shape · a parser-generator scales.

— NUANCES & GOTCHAS

- **Class explosion** — every rule is a class; complex grammars become unmanageable
- **Parsing is separate** — Interpreter defines & evaluates the AST; building it is the parser's job
- **Big grammar? use a parser** — ANTLR / PEG beat a hand-rolled class tree
- **The AST is a Composite** — terminals are leaves, nonterminals hold children
- **Add operations via Visitor** — don't bloat nodes with print / type-check / optimize
- **Performance** — tree-walking is slow; compile or cache hot expressions

— INTERVIEW PITFALLS

Q "When is Interpreter appropriate?"
Small, stable, well-understood grammars where a class-per-rule tree stays maintainable and speed isn't critical.

Q "Interpreter vs a parser generator?"
Interpreter hand-models & evaluates a tiny grammar; ANTLR / PEG generate efficient parsers for real languages.

Q "How does it relate to Composite & Visitor?"
The AST is a Composite; Visitor adds new operations over the nodes without changing them.

Q "A real-world example?"
Regular-expression matchers and rules / specification engines are interpreters of a small grammar.

— WATCH OUT

⚠ **Anti-pattern:** Skip it for anything but a small, stable grammar — class-per-rule explodes and tree-walking is slow. For real languages use a parser generator (ANTLR, PEG); for ad-hoc logic, plain code.

⚡ **Modern take:** rarely hand-written now — regex engines, rules / specification engines and query DSLs are the survivors; reach for ANTLR / PEG or an existing engine instead of building one.